

Casting convertible_to<bool> considered harmful

Document #: P1964R0
Date: 2019-11-15
Project: Programming Language C++
LEWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

[1 Abstract](#)

[2 Revision history](#)

[3 Problem statement](#)

[3.1 Nonuniformity of application remains](#)

[3.2 Contradiction with LWG's preferred direction for \[LWG2114\]](#)

[3.3 Conflict with existing practice](#)

[3.4 Unreasonable burden for implementers and users](#)

[4 Proposal: require the logical operators to “just work”](#)

[5 Alternative direction: limit the set of permitted types to a known set of well-behaved types \(not proposed\)](#)

[6 References](#)

§ 1 Abstract

[[P1934R0](#)] proposes removing the boolean concept and replacing its uses with convertible_to<bool> instead. The proposed replacement fails to completely fix the problems with boolean identified by the paper, contradicts LWG's preferred direction for [[LWG2114](#)], conflicts with existing practice, imposes an unreasonable burden on implementers and users alike, and should be reconsidered.

§ 2 Revision history

- R0: Initial revision for the post-Belfast mailing. An early draft of this paper was written during the Belfast meeting, attached to the LEWG wiki, and subsequently presented to LEWG; this version has been substantially revised from that draft to better follow the actual presentation.

§ 3 Problem statement

[[P1934R0](#)] proposes removing the boolean concept and replacing its uses with convertible_to<bool> instead. As boolean is currently used in the comparison concepts and the predicate concept, this change

requires generic code performing comparisons and invoking predicates to convert the result to `bool`. This can be done implicitly, explicitly, or contextually:

Implicit	Explicit	Contextual
<pre>bool r = x != y; bool f() { /* ... */ return a < b; }</pre>	<pre>static_cast<bool>(x == y) bool(c > d)</pre>	<pre>if (x == y) { /* ... */ } while (pred(z)) { /* ... */ }</pre>

However, even though the built-in operators `!`, `&&` and `||` contextually convert their operands to `bool`, the existence of operator overloading means that they cannot be used directly with something that's only required to model `convertible_to<bool>` is required:

Cast not required	Cast required
<pre>template<input_iterator I, sentinel_for<I> S, weakly_incrementable O> requires indirectly_copyable<I, O> O copy(I first, S last, O out) { for(; first != last; ++first, (void) ++out) { *out = *first; } return out; }</pre>	<pre>template<input_iterator I1, sentinel_for<I1> S1, input_iterator I2, sentinel_for<I2> S2> requires indirectly_comparable<I1, I2, ranges::equal> bool equal(I1 first1, S1 last1, I2 first2, S2 last2) { for(; bool(first1 != last1) && bool(first2 != last2); // both casts required ++first1, (void) ++first2) { if (*first1 != *first2) return false; } return bool(first1 == last1) && bool(first2 == last2); // ditto }</pre>

There are a multitude of problems with this proposed replacement.

§ 3.1 Nonuniformity of application remains

The keenly-eyed reader may have noticed that the examples in the above table are the same as those in section 2.2 “Nonuniformity of application” of [\[P1934R0\]](#), only modified to be correct.

In other words, P1934R0 fails to solve the motivating example it presented for this problem: a cast is still sometimes required, sometimes not. While it is certainly an improvement over `boolean` - the demarcation of the two “sometimes” is cleaner and easier to reason about - the problem remains.

§ 3.2 Contradiction with LWG’s preferred direction for [\[LWG2114\]](#)

Since C++98, the current library specification has been filled with requirements for something to be “convertible to `bool`” requirements (see, e.g., what is now called *Cpp17LessThanComparable* 16.5.3.1 [\[utility.requirements\]](#)) and “contextually convertible to `bool`” has been added to the mix since C++11 (see, e.g., *Cpp17NullablePointer* 16.5.3.3 [\[nullablepointer.requirements\]](#)).

However, these requirements date from a time where library specification has been quite imprecise; as Casey Carter put it during the LEWG discussion of an early draft of this paper, what was meant was more like “it converts to `bool` when the library wants it to convert to `bool`”.

A library issue, [\[LWG2114\]](#), was opened in 2011 to address the formulation of these requirements; in 2012, STL explained in the issue that implementations want to do things well beyond just converting them to `bool`; the example given from the Dinkumware Standard Library implementation then were:

```
if (pred(args))
if (!pred(args))
if (cond && pred(args))
if (cond && !pred(args))
```

All but the first would have required explicit `bool` casts if “convertible to `bool`” were the sole requirement.

LWG’s direction for LWG2114 has consistently been to require the logical operators to work correctly for these types. However, formulation of correct wording has proven difficult, and since this was seen as a defect in wording and required no implementation changes, it justifiably received a lower priority.

[\[P1934R0\]](#) contradicts this longstanding direction. If the former is adopted, then either we need to require implementations to litter `bool` casts everywhere through their existing code, or introduce an odd inconsistency in how the algorithms handle such types. Neither is appealing.

§ 3.3 Conflict with existing practice

[\[P1934R0\]](#) claims that `convertible_to<bool>` “is quite close to the ‘old’ notion of what a predicate should yield”. As seen from the history above, this is simply not the case in practice. None of the three major implementations in fact accept everything “convertible to `bool`”; they all make use of logical operations on the result and expect them to work. It takes five minutes to find examples in their current code base:

Implementation	Example
MSVC	<code>while (_UFirst1 != _ULast1 && _UFirst2 != _ULast2 && _Pred(*_UFirst1, *_UFirst2))</code>
libstdc++	<code>while (__first != __last && !__pred(__first))</code>
libc++	<code>for (; __first1 != __last1 && __first2 != __last2; ++__first1, (void) ++__first2)</code>

It’s also worth noting that two of the three examples above depend on the built-in operator `&&`’s short-circuiting behavior.

In other words, while “convertible to `bool`” or “contextually convertible to `bool`” might have been the requirement *on paper*, it has never been the reality.

§ 3.4 Unreasonable burden for implementers and users

The benefit conferred by the choice of `convertible_to<bool>` is that it enables certain highly questionable types to be returned from predicates and comparisons - overloading `operator&&` and `operator||` is universally recommended against due to such overloads not having the built-in operator’s short-circuit semantics. But the costs imposed are substantial enough to be unreasonable when measured against the minimal benefit it confers:

- Implementations, as well as generic code authors using standard library concepts, must now perform `bool` casts for correctness. The fact that these casts are only *sometimes* required compounds the problem.
- It’s unclear how many will actually do so, given the unnatural verbosity of these casts, and the lack of these questionable types in the wild. In LWG discussion, a major implementation stated that they have never received a user complaint for not supporting such types, even as they added support for overloaded address-of and comma operators. The lack of such types in the wild also makes it easy to overlook the need for casting and difficult to test for missing casts.
- It is very common for authors of concrete iterators in particular to just specify the iterator concept (or named requirement) supported. The standard itself does so for the iterators of containers, as well as things like `filesystem::directory_iterator`. Without further changes in the library specification, *users* would be required to cast the result of comparing two `vector<int>::iterator` to `bool`, which is obviously untenable. Similarly, authors of concrete iterators will need to revise their documentation to assure their users that casting is not required. While this might not be a major burden, the fact that doing so is necessary at all should give us pause.

§ 4 Proposal: require the logical operators to “just work”

Instead of `convertible_to<bool>`, we propose to replace `boolean` with an exposition-only concept `boolean-testable`:

```
template<class T>
concept boolean-testable = convertible_to<T, bool> && requires (T&& t) {
    { !std::forward<T>(t) } -> convertible_to<bool>;
};
```

and further add the semantic requirement that all logical operators “just work”:

- `!` can be overloaded but must have the usual semantics;
- `&&` and `||` must have their built-in meaning.

This is the same set of operations required to be supported by the current proposed resolution of [LWG2114]; casting to `bool` will not be necessary for these operations.

The primary difficulty that has plagued previous attempts at expressing the requirement as to `&&` and `||` is that it seems to require universal quantification: for a *boolean-testable* type, we must require that `&&` and `||` can be used with *every* *boolean-testable* type. This is impossible to express in concepts, and exceedingly difficult at best to formulate even in prose, as the drafting history of [LWG2114] attests. Moreover, stating the requirement in this manner makes it difficult to answer even simple questions like “is `bool` *boolean-testable*?” If there’s a `Evil` type that works with itself but no other type, is `Evil` the broken one, or is it everything else? How can you tell?

To address this difficulty and allow the type to be analyzed in isolation, we propose to strengthen the requirement: the type must not introduce a potentially viable `operator&&` or `operator||` candidate into the overload set. This is a requirement that can be answered easily: given a type, we know its member functions and its associated namespaces and classes (if any). We therefore know the result of class member lookup and argument-dependent lookup for the names `operator&&` and `operator||`. If there is no member with these names, and ADL also does not find an overload that can possibly be viable, then we know that using this type in an expression cannot possibly bring in viable `operator&&` and `operator||` overloads, and so using `&&` or `||` with two such types will always resolve to the built-in operator.

While this is a stronger requirement than what is strictly necessary, it allows analysis based on a single type (possibly by a static analyzer), and still admits a wide set of models including well-behaved class and enumeration types. There is some subtlety here (the wording needs to be carefully crafted to ensure that `std::valarray`’s `operator&&` and `operator||` do not disqualify `std::true_type`, for example), but the rule to teach is simple: just don’t overload the conditional operators, and your type will be fine.

§ 5 Alternative direction: limit the set of permitted types to a known set of well-behaved types (not proposed)

Alternative suggestions have been made to limit these expression to a small set of permitted types that we know to be well-behaved. Various options have been proposed in this direction.

Option	Permitted result type of a comparison of predicate
<code>same_as<bool></code>	<code>bool</code> , only. <code>const bool&</code> is not allowed.
<code>decays_to<bool></code>	Anything that decays to <code>bool</code> , thus allowing, e.g., <code>const bool&</code> (important for when a pointer to data member is used as an invocable predicate)
<code>decays_to<integral></code>	Anything that decays to an integral type, such as <code>bool</code> , <code>int</code> , or <code>const int&</code> . This allows things like Windows <code>BOOL</code> and C functions like <code>isupper</code> that returns an <code>int</code> .
<code>decays_to<integral or pointer or pointer-to-member></code>	Anything that decays to an integral/pointer/pointer to member type. This enables returning possibly-null

Option	Permitted result type of a comparison of predicate
	pointers from predicates without having to convert them to <code>bool</code> first.
<code>decays_to<integral or true_type or false_type></code>	This accepts two known-good class types for which support have been requested.

By necessity, they are all significantly more limiting than the *boolean-testable* approach: class types and enumeration types cannot be supported generally; even supporting a select set requires giving up support for pointers and pointers to member. This paper does not propose this approach, but mentions it for completeness.

§ 6 References

[LWG2114] Daniel Krüger. Incorrect “contextually convertible to bool” requirements.

<https://wg21.link/lwg2114>

[P1934R0] Casey Carter, Christopher Di Bella, Eric Niebler. 2019. boolean Considered Harmful.

<https://wg21.link/p1934r0>